

Phase 12 – Monitoring & Observability on Amazon EKS

Phase Overview

After building workloads, networking, storage, autoscaling, namespaces, quotas, and StatefulSets, the next logical question becomes:

How do I know when something is broken?

In production environments, engineers do not continuously run:

```
kubectl get pods  
kubectl describe pod  
kubectl logs
```

to monitor hundreds of applications.

Instead, organizations use monitoring and observability platforms that continuously collect metrics, store them, visualize them, and generate alerts.

In this phase, we deployed a complete monitoring stack on EKS using:

- Prometheus
- Grafana
- AlertManager
- Node Exporter
- Kube State Metrics

Learning Objectives

By the end of this phase you will understand:

How Kubernetes metrics are collected

How Prometheus stores metrics

How Grafana visualizes metrics

How to access Grafana

How to generate load

How HPA uses CPU metrics

How to troubleshoot metric issues

How labels can impact HPA behavior

Real World Problem

Imagine your production environment contains:

50 Applications / 20 Worker Nodes / 500 Pods

When an application becomes slow:

Is CPU high?

Is memory exhausted?

Did HPA scale?

Did a node become unhealthy?

Did storage fill up?

Without monitoring: Guesswork

With monitoring: Data-driven troubleshooting

Monitoring Architecture

What we built:

Kubernetes Cluster



Node Exporters



Kube State Metrics



Prometheus



Grafana



Engineer

Component Explanation

Prometheus

Purpose:

Collect Metrics

Store Metrics

Query Metrics

Examples:

CPU Usage

Memory Usage

Pod Counts

Node Health

Network Usage

Grafana

Purpose:

Visualization Layer

Provides:

Dashboards

Charts

Graphs

Historical Trends

AlertManager

Purpose:

Generate Alerts

Examples:

Node Down

High CPU

Memory Pressure

Pod Failures

[Node Exporter](#)

Purpose: Collect Node Metrics

Examples:

CPU

Memory

Disk

Filesystem

Network

[Kube State Metrics](#)

Purpose: Collect Kubernetes Object Metrics

Examples:

Deployments

Pods

StatefulSets

PVCs

Services

HPA

Step A – Create Monitoring Namespace

Why?

Monitoring components should be isolated from applications.

Production clusters typically have:

default

dev

prod

monitoring

kube-system

Create Namespace

```
kubectl create namespace monitoring
```

Verify:

```
kubectl get ns
```

Expected:

monitoring

Step B – Add Prometheus Helm Repository

Why?

Helm repositories act like package repositories.

Similar to:

apt

yum

chocolatey

for Kubernetes.

Add Repository

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
```

Expected:

```
"prometheus-community" has been added
```

Update Repositories

```
helm repo update
```

Expected:

```
Update Complete
```

Step C – Install Monitoring Stack

Instead of installing components individually, we use:

```
kube-prometheus-stack
```

This installs:

Prometheus

Grafana

AlertManager

Node Exporter

Kube State Metrics

Prometheus Operator

Install

```
helm install monitoring prometheus-community/kube-prometheus-stack \
  --namespace monitoring
```

Expected:

```
STATUS: deployed
```

Step D – Verify Installation

Check Pods

```
kubectl get pods -n monitoring
```

Expected:

```
alertmanager-monitoring...
```

```
monitoring-grafana...
```

```
monitoring-kube-state-metrics...
```

```
monitoring-prometheus-node-exporter...
```

```
prometheus-monitoring...
```

All should eventually become:

Running

Step E – Verify Services

Check Services

```
kubectl get svc -n monitoring
```

Expected:

```
monitoring-grafana
```

```
monitoring-kube-prometheus-prometheus
```

```
monitoring-kube-state-metrics
```

```
alertmanager
```

Notice: TYPE = ClusterIP

Meaning:

Accessible inside cluster

NOT accessible from internet

Step F – Retrieve Grafana Password

Grafana stores credentials inside Kubernetes Secrets.

Get Password

```
kubectl --namespace monitoring get secrets monitoring-grafana \
-o jsonpath="{.data.admin-password}"
```

Output:

Base64 Encoded Password

Decode Password (PowerShell)

```
[System.Text.Encoding]::UTF8.GetString(
[System.Convert]::FromBase64String("<encoded-password>")
)
```

Step G – Access Grafana

Port Forward

```
kubectl port-forward -n monitoring svc/monitoring-grafana 3000:80
```

Leave terminal running.

Open Browser

<http://localhost:3000>

Login:

Username:

admin

Password:

decoded-password

Step H – Explore Dashboards

Navigate:

Dashboards

↓

Browse

Preloaded dashboards include:

Kubernetes / Compute Resources / Cluster

Kubernetes / Compute Resources / Pod

Kubernetes / API Server

CoreDNS

AlertManager

Cluster Dashboard Concepts

Important panels:

CPU Utilization

CPU Requests

CPU Limits

Memory Utilization

Memory Requests

Memory Limits

Why Requests Matter

Earlier phases introduced:

resources:

requests:

cpu: 100m

limits:

cpu: 200m

Grafana visualizes these values across the cluster.

Step I – Generate CPU Load

Purpose

Validate monitoring stack.

Connect To Nginx Pod

```
kubectl get pods
```

Example:

```
nginx-deployment-674fb86454-k2c9k
```

Enter Pod

```
kubectl exec -it nginx-deployment-674fb86454-k2c9k -- sh
```

Generate CPU Load

```
while true; do :; done
```

This creates:

Continuous CPU Consumption

Step J – Observe Metrics

Metrics Server

```
kubectl top pods
```

Observed:

Normal Pods:

8m CPU

Loaded Pod:

200m CPU

This proved:

Metrics Server Working

CPU Load Generation Successful

Step K – Observe Grafana

Navigate:

Kubernetes / Compute Resources / Pod

Watch:

CPU Usage

for the loaded pod.

Expected:

CPU spike visible

Real Troubleshooting Scenario Encountered

During the lab:

`kubectl describe hpa nginx-deployment`

showed:

`cpu: <unknown>/25%`

instead of:

`cpu: 39%/25%`

Investigation

Error:

`failed to get cpu utilization:`

`missing request for cpu in container nginx of Pod web-0`

Root Cause

Deployment Pods:

`app=nginx`

StatefulSet Pods:

`app=nginx`

Both workloads shared the same label.

Deployment Labels

app: nginx

StatefulSet Labels

app: nginx

Result:

Label Collision

HPA evaluated StatefulSet pods which lacked CPU requests.

Why HPA Failed

HPA calculates:

CPU Utilization

Current CPU Usage

÷

CPU Request

Example:

$50m \div 100m = 50\%$

Without CPU requests:

Division Impossible

Result:

<unknown>/25%

Remediation

Changed StatefulSet labels from:

app: nginx

to:

app: web

StatefulSet Fix

```
selector:
  matchLabels:
    app: web
```

```
template:
  metadata:
    labels:
      app: web
```

Headless Service Fix

```
selector:
  app: web
```

Recreate StatefulSet

Delete:

```
kubectl delete statefulset web
```

Reapply:

```
kubectl apply -f headless-service.yaml
```

```
kubectl apply -f statefulset-storage.yaml
```

Verify:

```
kubectl get pods --show-labels
```

Expected:

```
web-0 app=web
```

```
web-1 app=web
```

```
web-2 app=web
```

Verification

Run:

```
kubectl describe hpa nginx-deployment
```

Before:

cpu: <unknown>/25%

After:

cpu: 39% (39m) / 25%

Success:

ScalingActive = True

ValidMetricFound

Key Lessons Learned

Monitoring Is Essential

Without monitoring:

Guessing

With monitoring:

Data

HPA Depends On Requests

Always define:

resources:

requests:

cpu:

Otherwise HPA cannot calculate utilization.

Labels Matter

One reused label caused:

HPA Confusion

Metric Collection Issues

Troubleshooting Overhead

Prometheus + Grafana = Production Standard

These tools provides;

Metrics Collection

Visualization

Troubleshooting

Capacity Planning

Alerting

are among the most common monitoring solutions used in Kubernetes environments.